DESARROLLADOR DE APLICACIONES

Técnico Superior en Desarrollo de Aplicaciones Multiplataforma

0484 MP Bases de Datos
UD3 Realización de consultas

Índice

01. Introducción

1.1. Consola de MySQL
1.2. MySQL Workbench
1.3. Instalar base de datos ejemplo
1.4. Notación de la sintaxis

02. Sentencia SELECT

2.1. Consultas básicas
2.2. Eliminación de filas repetidas: cláusulas ALL, DISTINCT
2.3. Ordenación de filas: cláusula ORDER BY
2.4. Limitar el número de filas: cláusula LIMIT
2.5. Selección de filas: cláusula WHERE
2.6. Agrupación de resultados: funciones de agregado y cláusula GROUP BY
2.7. Filtro de grupos: cláusula HAVING

03. Subconsultas

3.1. Predicados de comparación
3.2. Predicado de pertenencia a un conjunto: IN
3.3. Predicados cuantificados: ALL y ANY
3.4. Predicados de existencia: EXISTS

04. Consultas correlacionadas

05. Consultas con tablas derivadas

06. Consultas sobre varias tablas: JOIN

6.1. Composición interna: INNER JOIN

6.2. Composición externa: OUTER JOIN

07. Bibliografía

01. Introducción

Como se comentó en temas anteriores el lenguaje de manipulación de datos (DML, Data Manipulation Language) incluye las instrucciones que me van a permitir manipular el contenido de la base de datos, es decir me va a permitir la inserción de datos nuevos en las tablas o vistas, eliminación y actualización de esos datos o consulta de la información contenida en la base de datos. Por lo tanto, las sentencias DML del lenguaje SQL son:

- SELECT: permite la consulta de información almacenada en la base de datos.
- INSERT: permite insertar registros en alguna tabla de la base de datos.
- DELETE: borra registros de una tabla de la base de datos.
- UPDATE: modifica los registros de una tabla de la base de datos.

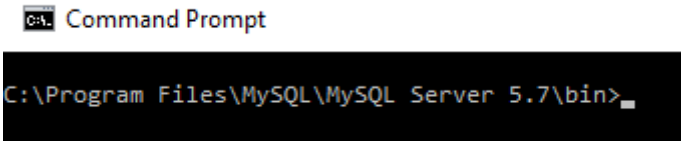
En este tema 3 aprenderemos a realizar consultas de diversa complejidad sobre una base de datos, por lo tanto, estudiaremos en profundidad la sentencia SELECT. En el siguiente tema, 4, se estudiará el resto de sentencias del lenguaje DML, (INSERT, UPDATE y DELETE). Trabajaremos con el SGBD MySQL para la realización de todos los ejemplos y ejercicios tanto de este como de posteriores temas.

Lo primero que debemos hacer es instalar MySQL en nuestro ordenador descargando para ello la última versión disponible en su página web oficial. La instalación por defecto de MySQL en cualquier versión de Windows es hoy en día muy sencilla ya que está perfectamente empaquetada en un instalador (MySQL Installer) con un asistente que nos guiará durante todo el proceso de instalación.

El instalador de MySQL (MySQL Installer) incluye varios productos entre los que cabe destacar MySQL Server y MySQL Workbench. Este último es una herramienta con interfaz gráfica que nos va a permitir realizar de forma mucho más sencilla y visual cualquier acción sobre la base de datos. Todas las sentencias que ejecutemos en MySQL Workbench se pueden realizar del mismo modo en la consola de comandos de MySQL, pero al ser una herramienta con una interfaz gráfica mucho más atractiva que la consola, nos resultará más sencillo encontrar posibles errores en nuestros comandos, así como interpretar los resultados.

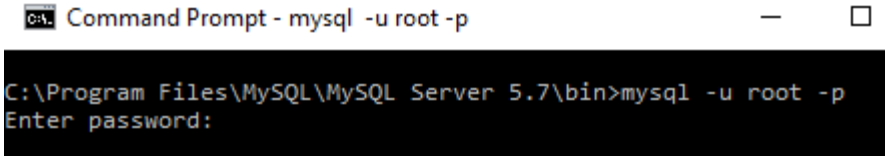
1.1. Consola de MySQL

La consola o intérprete de comandos de MySQL tiene como misión enviar los comandos al SGBD y mostrar los resultados devueltos por pantalla. Para acceder a esta consola de MySQL en Windows, habrá que situarse, desde una terminal, en la carpeta bin donde se instaló MySQL, (normalmente C:\Program Files\MySQL\MySQL Server 5.7\bin).
Una vez en este punto, tan solo habrá que escribir el nombre (mysql) con ciertas opciones, por ejemplo:



mysql -u root -p

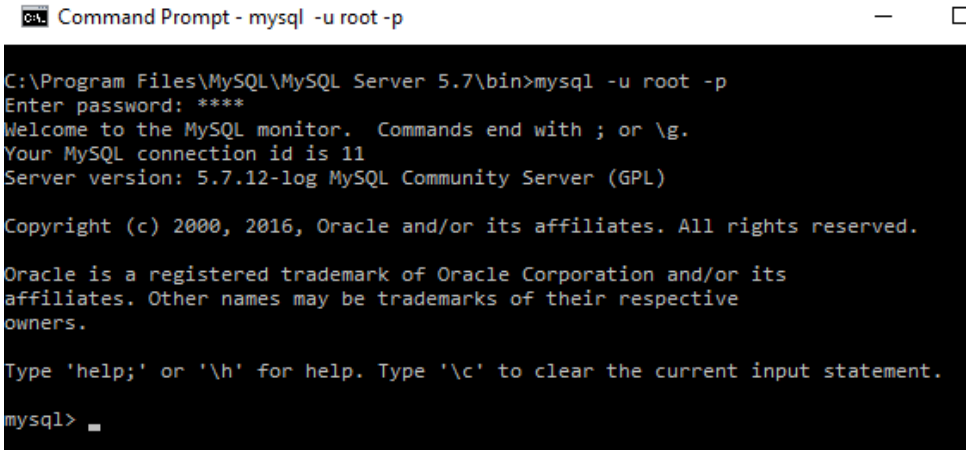
donde -u root indica que me conectaré con el usuario root, y la opción -p indica que me solicite el password.



Para ver todas las opciones posibles, ejecute:

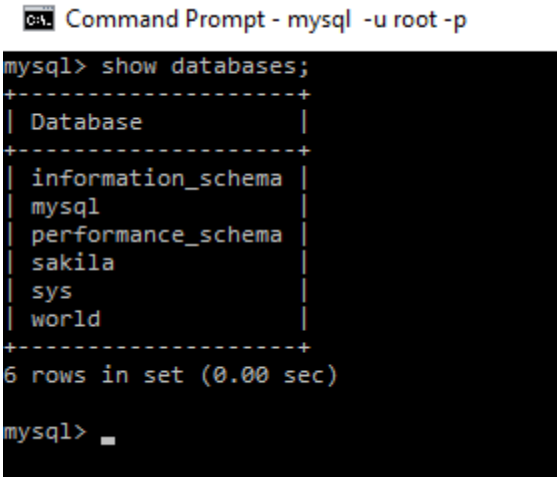
mysql --help

Una vez introducida la contraseña, se mostrará una información de accesos seguida del prompt de mysql (mysql>). En este punto puedo introducir cualquier comando SQL.



Por ejemplo, puedo consultar todas las bases de datos que tiene mi SGBD en este momento mediante el comando:

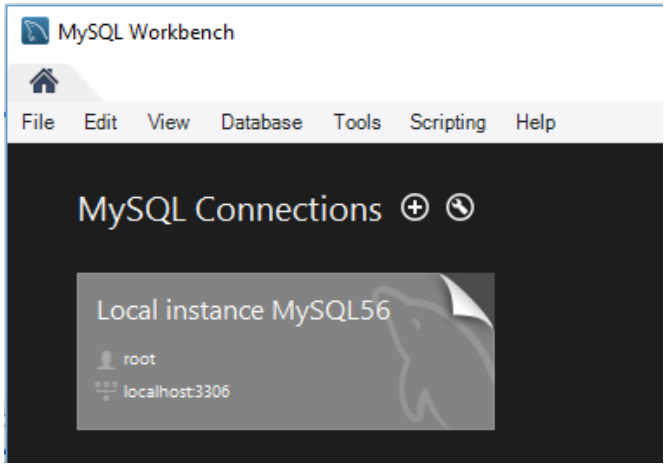
show databases;



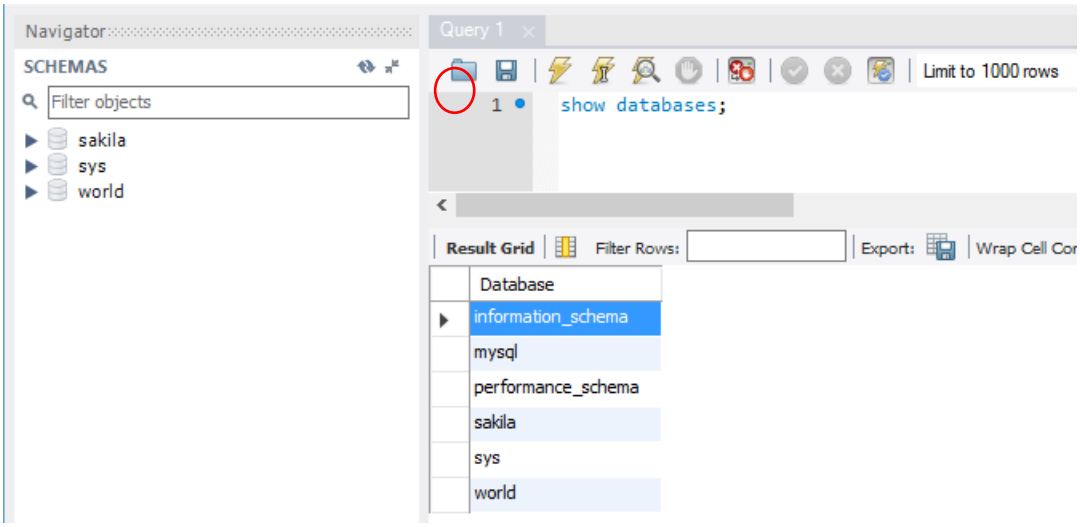
1.2. MySQL Workbench

MySQL Workbench es una herramienta con una interfaz gráfica amigable que permite realizar el diseño, la creación y el mantenimiento de la base de datos, además de la manipulación de sus datos. Como se comentó anteriormente, cualquier comando que ejecute en el Workbench puede ser a su vez ejecutado en la consola de MySQL pero de una manera menos atractiva.

Para ejecutar el comando "show databases;" en Workbench, tan solo tengo que abrir el programa, conectarme al SGBD a través de una de las conexiones que probablemente haya configurado en el proceso de instalación, me pedirá introducir la contraseña, y en la pantalla que me aparece ya podré poner mi comando SQL.



Una vez escrito el comando SQL, para ejecutarlo, pulsaré el botón Ejecutar que se simboliza con un rayo amarillo como se muestra en la siguiente imagen.

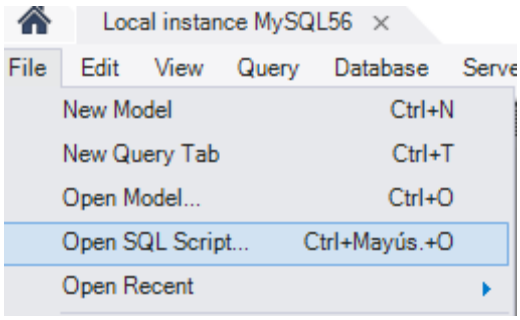


1.3. Instalar base de datos ejemplo

Para la explicación de los diversos comandos, así como para la realización de los ejercicios se usará la base de datos Centro de estudios que se proporciona en el archivo centroestudios.sql.

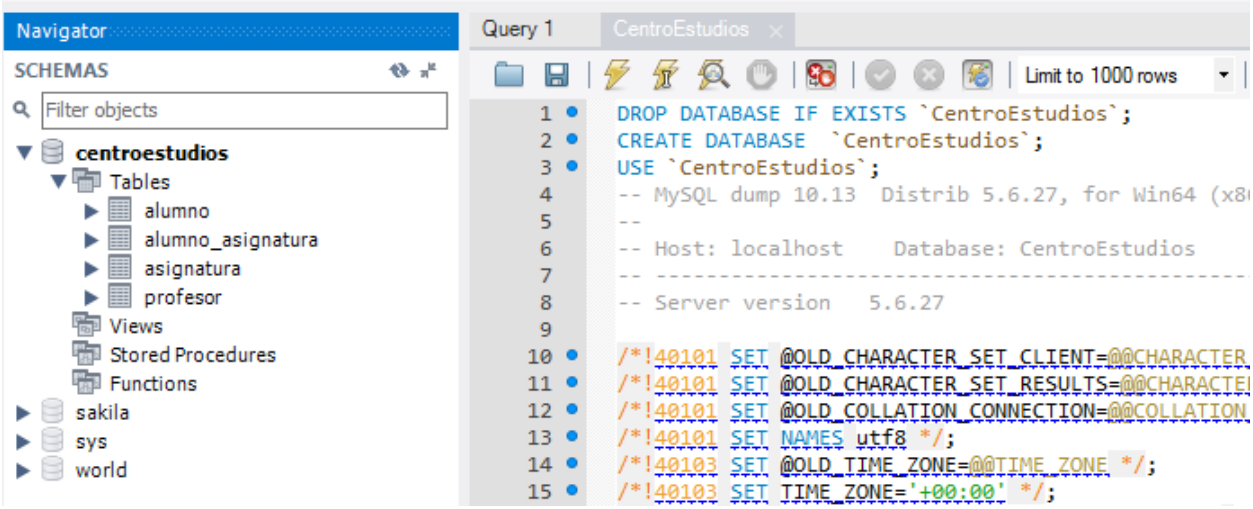
Para importar a MySQL esta base de datos realiza los siguientes pasos:

- En Workbench, accede a la opción "Abrir Script SQL" del menú "Archivo".

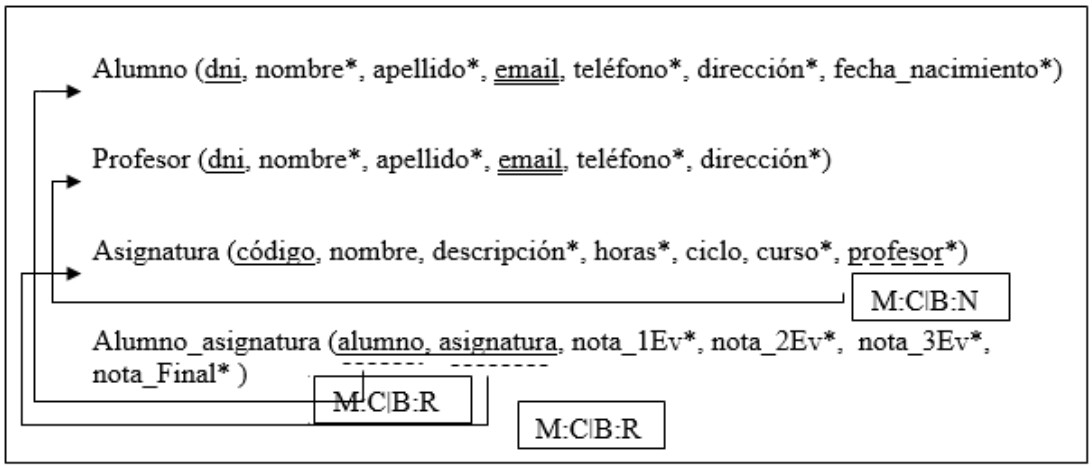


- Selecciona el archivo centroestudios.sql en la carpeta correspondiente de tu equipo.

- Una vez abierto el archivo, simplemente tendré que ejecutar esta script, pulsando el botón ejecutar (rayo amarillo).
- Esto creará la base de datos CentroEstudios y le añadirá unos datos iniciales. Puedo comprobar en la ventana "Esquemas" (en la columna izquierda) que la base datos ha sido instalada correctamente.



A continuación, se muestra el modelo entidad relación de esta base de datos Centro de estudios, con el fin de que se entienda perfectamente toda su estructura.



Como podemos tener varias bases de datos creadas en nuestro SGBD, antes de realizar cualquier operación sobre una base de datos en concreto, tenemos que indicarle a nuestro SGBD con cual de todas las bases de datos voy a trabajar. Por

ejemplo, si voy a trabajar con la base de datos Centro de estudios, tenemos que poner el comando:

use centroestudios;

Es decir, escribiré el comando "use" seguido del nombre de la base de datos que vamos a utilizar. Con esto le indico al SGBD que cuando por ejemplo utilice la tabla alumno, me estaré refiriendo a la tabla alumno de la base de datos centro de estudios (ya que puede existir una tabla llamada alumno en otra base de datos que esté instalada en mi SGBD).

Como se observa en la imagen anterior MySQL Workbench señala la base de datos activa en negrita en la ventana "esquemas" situada, por defecto, en la parte izquierda.

1.4. Notación de la sintaxis

La nomenclatura que usaremos de ahora en adelante para expresar la sintaxis de los diferentes comandos es:

- Palabras en mayúsculas: son palabras reservadas por el lenguaje que se utilizarán para un objetivo específico, por ejemplo, SELECT, CREATE, INSERT, UPDATE, etc. Como ocurre en cualquier lenguaje de programación, no puedo definir objetos (tablas, atributos, vistas, etc) con esos nombres.
- Cochetes, []: indica opciones. Todo lo que va entre corchetes es opcional, es decir, será una cláusuala que puede incorporarse a la sentencia o no dependiendo de lo que el programador desee significar.
- Llaves, {}: indica opciones obligatorias, es decir, debe escogerse una de las opciones que se encuentran dentro de las llaves.
- *Pipe*, |: sirve para distinguir entre las distintas opciones que se encuentran bien entre corchetes o bien entre llaves.
- Puntos suspensivos, …: indica puede repetirse varias veces el último elemento sintáctico.

02. Sentencia SELECT

La sentencia SELECT se utiliza para consultar información almacenada en las distintas tablas de una base de datos. Es una sentencia muy potente, probablemente la sentencia más compleja de SQL, compuesta por diversas cláusulas, que me van a permitir realizar desde consultas sencillas, hasta consultas muy complicadas en las que están en involucradas varias tablas.

A continuación, se muestra la sintaxis de la sentencia SELECT con prácticamente todas sus cláusulas, recordemos que los corchetes marcan opcionalidad y de ahí la gran versatilidad de esta sentencia.

```
SELECT [ALL | DISTINCT] expresión [, expresión...]
[
FROM tablas
[WHERE condiciones]
[GROUP BY { nombre_columna | expresión | posición } [ASC | DESC ] ,... [WITH
ROLLUP]]
[HAVING condiciones]
[ORDER BY { nombre_columna | posición_columna} [ASC | DESC ] ,...]
[LIMIT [desplazamiento, ] num_filas ]
];
```

En los siguientes apartados se irán explicando poco a poco cada una de estas cláusulas, partiendo del comando más simple que se puede ejecutar con esta sentencia SELECT, hasta ser capaz, al final del tema de ejecutar consultas complejas en la que estarán involucradas todas sus cláusulas.

Es importante destacar que el resultado de la ejecución de la sentencia SELECT sobre una o varias tablas es siempre otra tabla. Las columnas de la tabla resultante serán las que se enumeren en la cláusula SELECT, y en el mismo orden.

2.1. Consultas básicas

La sintaxis mínima de una sentencia SELECT es la siguiente:

```
SELECT expr [, expr...]
[FROM tabla];
```

Donde:

expr: nombre_columna [AS alias] | * | expresión [AS alias]

A continuación, explicaremos y mostraremos ejemplos de cada una de estas partes:

- FROM tabla: indica la tabla (o tablas) afectada en la consulta.
- Nombre_columna: indica el atributo (columna) de la tabla indicada en la cláusula FROM del que se desea mostrar su información. Se puede mostrar el contenido de una sola columna o de varias.

Ejemplo: Muesta el email de los alumnos de la base de datos centro de estudios.

```
SELECT email FROM alumno;
```

Ejemplo: Muesta el nombre, apellido y email de los alumnos registrados en la base de datos.

```
SELECT nombre, apellido, email FROM alumno;
```

- *: si deseo mostrar el contenido de todas las columnas, no es necesario que escriba todos sus nombres, la opción * es equivalente a incluir todas las columnas de la tabla indicada en la cláusula FROM.

Ejemplo: Muesta todos los datos de los alumnos registrados en la base de datos centro de estudios.

```
SELECT * FROM alumno;
```

- Expresión: la información que se desea mostrar puede ser, en lugar de simplemente el valor de una columna, una expresión en la que esté involucrada alguna columna de la tabla incluida en la claúsula FROM. Estas expresiones pueden ser algebráicas(+, -, *, /, ^) o funciones propias de MysQL (de fecha como CURRENT_DATE o DATE_ADD, de cadenas de

caracteres como UPPERCASE, LOWERCASE o LENGHT, numéricas como ABS, SQRT o MOD, etc).

Ejemplo: Muesta el nombre, apellido e email de los alumnos registrados en la base de datos centro de estudios, de tal manera que el nombre y apellido de cada alumno se muestre en una sola columna. Para ello utilizaré la función de MySQL "CONCAT" que me permitirá concatenar varios campos en un solo.

```
SELECT CONCAT(nombre, apellido), email FROM alumno;
```

Ejemplo: Muesta la suma de 3+5 y el producto de 7*8. Como se observa en este ejemplo, la clásula FROM también es opcional, en este caso las operaciones que se piden realizar no tienen involucrados ninún atributo de ninguna base de datos, por lo que no necesitaré poner la cláusula FROM.

```
SELECT 3+5, 7*8;
```

- AS: permite especificar un alias para una columna o expresión. Si el alias tiene espacios en blanco, debe escribirse entre comillas.

Ejemplo: modifica ejemplo anterior en el que se pide mosrtar el nombre, apellido e email de los alumnos registrados en la base de datos centro de estudios, mostrando el nombre y apellido de cada alumno en una sola columna, para que el nombre de dicha columna sea "Nombre y apellido"

```
SELECT CONCAT(nombre, apellido) AS 'Nombre y apellido', email FROM alumno;
```

O lo que es lo mismo, sin usar la palabra reservada AS:

```
SELECT CONCAT(nombre, apellido) 'Nombre y apellido', email FROM alumno;
```

```
SELECT CONCAT(nombre, apellido), email
FROM alumno;
```

Result Grid | Filter Rows: | Export:

CONCAT(nombre, apellido)	email
AnaSanz	ana.sanz@gmail.com
BlancaCarrillo	blanca.carrillo@yahoo.es
AnaPuerto	ana.puerto@gmail.com

```
SELECT CONCAT(nombre, apellido) AS 'Nombre y apellido', email
FROM alumno;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Nombre y apellido	email
AnaSanz	ana.sanz@gmail.com
BlancaCarrillo	blanca.carrillo@yahoo.es
AnaPuerto	ana.puerto@gmail.com

Del mismo modo, pueden utilizarse alias para las tablas, lo que en consultas complicadas en las que intervengan varias tablas puede llegar a ser muy útil. A la hora de referenciar un atributo conviene poner antes del nombre de dicho atributo, el alias de la tabla seguido de un punto.

Ejemplo: el mismo ejercicio anterior creando un alias 'a' para la tabla alumno quedaría:

SELECT CONCAT(a.nombre, a.apellido) AS 'Nombre y apellido', a.email FROM alumno a;

2.2. Eliminación de filas repetidas: cláusulas ALL, DISTINCT

Las cláusulas ALL y DISTINCT me van a permitir eliminar filas repetidas en la tabla resultado generada con la ejecución de la consulta. La sintaxis de la sentencia SELECT añadiendo esta opción sería:

SELECT [ALL | DISTINCT] expr [, expr...]
[FROM tabla];

- ALL: si incluimos la opción ALL, se mostrarán todos los valores de la expresión (incluídas las repeticiones). Es la opción por defecto.
- DISTINCT: esta opción fuerza que solo se muestren los registros con valores distintos, es decir, elimina los valores repetidos.

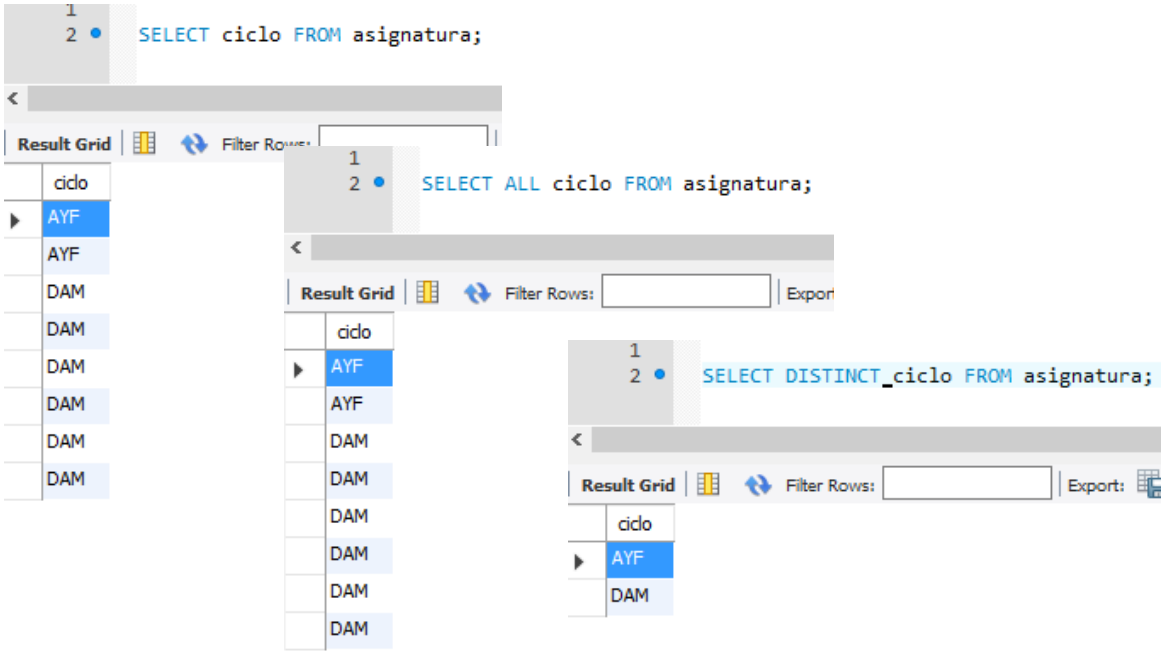
Ejemplo: Muesta todos los ciclos que tiene alguna asignatura registrada.

```
SELECT ALL ciclo FROM asignatura;
```

Ejemplo: Muesta todos los ciclos que tiene alguna asignatura registrada, de tal manera que no aparezcan ciclos repetidos.

```
SELECT DISTINCT ciclo FROM asignatura;
```

Observa, en las imagenes siguientes,la diferente ente estas tres opciones:



2.3. Ordenación de filas: cláusula ORDER BY

Podemos querer mostrar el resultado de la ejecución de la consulta en un orden determinado, para ello utilizaré la cláusula ORDER BY. Si no se especifica esta opción, las filas se devuelven en cualquier orden.

SELECT [ALL | DISTINCT] expr [, expr...]
[FROM tabla
[ORDER BY { nombre_columna | posicion_columna} [ASC | DESC] ,...]
];

Esta cláusula va a permite ordenar el resultado de la consulta de forma ascendente (ASC) o descendente (DESC) por una o varias columnas. Por defecto será ascendente.

La columna por la que se desea ordenar se puede expresar por el nombre de la columna o por la posición de la columna en la cláusula SELECT.

Ejemplo: Muesta el nombre, apellido y fecha de nacimiento de todos los alumnos ordenados alfabéitcamente por apellidos.

```
SELECT nombre, apellido, fecha_nacimiento FROM alumno
ORDER BY apellido ASC;
```

O lo que es lo mismo:

```
SELECT nombre, apellido, fecha_nacimiento FROM alumno
ORDER BY 2 ASC;
```

Ejemplo: Muesta todos los datos de los alumnos ordenados alfabéitcamente por apellidos (las dos consultas siguientes muestran el mismo resultado).

```
SELECT * FROM alumno
ORDER BY apellido;
```

```
SELECT * FROM alumno
ORDER BY 3;
```

Ejemplo: Muesta todos los datos de las asignaturas ordenadas por ciclo, y para asignaturas del mismo ciclo ordenadas por curso:

```
SELECT * FROM asignatura
ORDER BY 5 ASC, 6  ASC;
```

O lo que es lo mismo:

```
SELECT * FROM asignatura
ORDER BY ciclo ASC, curso  ASC;
```

Ejemplo: Muesta el nombre y el apellido de todos los alumnos ordenados descendentemente por fecha de nacimiento.

```
SELECT nombre, apellido FROM alumno
ORDER BY fecha_nacimiento DESC;
```

En este último ejemplo, no se puede poner en la cláusula ORDER BY la posición de la columna en vez del nombre ya que no es una columna mostrada, es decir, no aparece en la cláusula SELECT.

2.4. Limitar el número de filas: cláusula LIMIT

Si se quiere limitar el número de registros devueltos en una consulta utilizaremos la cláusula LIMIT.

```
SELECT [ALL | DISTINCT ] expr [, expr...]
[FROM tabla
[ORDER BY { nombre_columna | posicion_columna} [ASC | DESC ] ,...]
[LIMIT [desplazamiento, ] num_filas ]
];
```

Donde num_filas contiene el número de registros que devolverá la consulta y desplazamiento a partir de qué fila se empieza a contar. Si no se especifica desplazamiento, empezará a devolver desde la primera fila.

Ejemplo: Muesta el nombre y el apellido de los cinco alumnos más pequeños.

```
SELECT nombre, apellido
FROM alumno
ORDER BY fecha_nacimiento DESC
LIMIT 5;
```

Ejemplo: Muesta el nombre y el apellido de los cinco alumnos con mayor edad, pero sin incluir al mayor de todos.

```
SELECT nombre, apellido
FROM alumno
ORDER BY fecha_nacimiento ASC
LIMIT 1, 5;
```

2.5. Selección de filas: cláusula WHERE

Si no se quieren mostrar todos los registros, sino sólo los que cumplan ciertas condiciones se deberá utilizar la cláusula WHERE.

```
SELECT [ALL | DISTINCT ] expr [, expr...]
[FROM tabla
[WHERE condiciones]
[ORDER BY { nombre_columna | posicion_columna} [ASC | DESC ] ,...]
[LIMIT [desplazamiento, ] num_filas ]
];
```

Donde condiciones es una expresión que indica las características de deben cumplir las filas para ser seleccionadas y, por tanto, mostradas en el resultado de la consulta. Aquellas filas que no cumplan estas condiciones no formarán parte del resultado de la consulta.

Estas condiciones que aparecen en la cláusula WHERE están formadas por operadores, operandos y paréntesis, de tal manera que:

- Los operandos pueden ser:
  - nombres de columnas (por ejemplo, dni, teléfono, emil, nota_Final, etc.).
  - constantes (por ejemplo, el número entero 10, o el decimal 2.5, la cadena de caracteres 'CETYS', la fecha '2017-01-01')

  - expresiones, bien sean algebráicas por ejemplo, (2+3, o (nota_1Ev+nota_2_Ev+nota_3Ev)/3, etc) o expresiones con funciones propias de MySQL(CURRENT_DATE(), UPPERCASE(nombre), etc.).
- Los operadores pueden ser:
  - De comparación: <, <=, >, >=, =, <> o != (distinto), !> (no mayor que), !<(no menor que).
  - Operadores lógicos: AND, OR, NOT.
  - Predicados avanzados: IN, BETWEEN, IS NULL, LIKE.
- Los paréntesis me van a permitir alterar la prioridad de los operadores. Por ejemplo, la multiplicación tiene más prioridad que la resta, por lo tanto no es lo mismo 2+3*4 (cuyo resultado es 14) que (2+3)*4 (cuyo resultado es 20).

A continuación, se explica y se muestra ejemplos con cada uno de estos operadores.

OPERADORES DE COMPARACIÓN

Estos operadores como su propio nombre indica sirven para comparar dos operandos, de esta manera es posible comparar si un valor es mayor que otro, si es distinto, etc.

Los operadores de comparación son:

Mayor	>
Menor	<
Mayor o igual	>=
Menor o igual	<=
Igual	=
Distinto	<> o !=
No mayor que	!>
No menor que	!<

Ejemplo: Muesta el nombre de todas las asignaturas de primer curso junto al ciclo formativo al que pertence.

```
SELECT nombre, ciclo FROM asignatura
WHERE curso = 1;
```

Ejemplo: Muesta todos los datos de las asignaturas que tengan una carga lectiva de más de 200 horas.

```
SELECT * FROM asignatura
WHERE horas > 200;
```

Ejemplo: Muesta el nombre de todas las asignaturas del ciclo formativo de Desarrollo de Aplicaciones Multiplataforma (DAM), junto con el curso en el que se imparte. El resultado se deberá mostrar ordenado por curso, y las asignaturas del mismo curso se deberán mostrar ordenadas alfabéticamente.

```
SELECT nombre, curso FROM asignatura
WHERE ciclo = 'DAM'
ORDER BY curso, nombre;
```

Observa como en los diversos ejemplos las constantes numéricas no van entrecomilladas, pero el resto de constantes (fecha, cadena) deben de ir entre comillas simples.

OPERADORES LÓGICOS

Este tipo de operadores nos permitirán combinar varias expresiones simples, por ejemplo, mostrar todas las asignaturas del primer curso del ciclo DAM.

Los operadores lógicos son AND, OR y NOT. Los dos primeros operadores, AND y OR, se aplican a dos o más operandos, mientras que el tercero, NOT, se aplica a un solo operando. Cuando se utiliza el operador:

- AND: el resultado es "verdadero" si todos sus operandos son verdaderos.
- OR: el resultado es "verdadero" si al menos uno de sus operandos es verdadero.
- NOT: el resultado es "verdadero" si el operando sobre el que se aplica es falso.

Ejemplo: Muesta todos los datos de las asignaturas del primer curso del ciclo DAM.

```
SELECT * FROM asignatura WHERE curso=1 AND ciclo='DAM';
```

Ejemplo: Muesta todos los datos de las asignaturas del primer curso del ciclo DAM y del ciclo de Administración y Finanzas (AYF).

```
SELECT * FROM asignatura
WHERE curso=1 AND (ciclo='DAM' OR ciclo='AYF');
```

Ejemplo: Muesta el nombre y el ciclo de las asignaturas que no sean impartidas por el profesor 63254157C ordenados alfabéticamente por ciclo, y para asignaturas del mismo ciclo ordenadas por curso.

```
SELECT nombre, ciclo FROM asignatura
WHERE NOT (profesor='63254157C')
ORDER BY ciclo, curso;
```

PREDICADO IN

El operador de pertenencia a conjuntos, IN, me va a permitir comprobar si el resultado de una expresión está incluido en el conjunto de valores especificados entre paréntesis tras la palabra reservada IN. Su sintaxis es:

expresión [NOT] IN (valor1, valor2, ...)

Si se especifica NOT, el predicado es verdadero si no el valor que toma expresión no está incluido en el conjunto de valores especificados.

Ejemplo: Muesta todos los datos de las asignaturas del primer curso del ciclo DAM y del ciclo de Administración y Finanzas (AYF).

```
SELECT * FROM asignatura
WHERE curso=1 AND ciclo IN ('DAM', 'AYF');
```

Como se verá más adelante en el tema, en lugar de expresar los elementos del conjunto explícitamente con una lista de valores separados por comas, es posible especificar este conjunto mediante una sentencia SELECT que devuelva una sola columna.

PREDICADO BETWEEN

El operador de rango, BETWEEN, me va a permitir comprobar si el resultado de una expresión está incluido entre dos valores dados (ambos inclusive). Su sintaxis es:

Expresión_1 [NOT] BETWEEN expresión_2 AND expresión_3

Si se especifica NOT, el predicado es verdadero si el valor que toma expresión_1, no está en el rango entre expresión_2 y expresión_3.

Ejemplo: Muesta todos los datos de los alumnos nacidos en la década de los 90.

```
SELECT * FROM alumno.
WHERE YEAR(fecha_nacimiento) BETWEEN 1990 AND 1999;
```

Ejemplo: Si en este caso queremos mostrar todos los datos de los alumnos que no han nacido en la década de los 90.

```
SELECT * FROM alumno
WHERE NOT YEAR(fecha_nacimiento) BETWEEN 1990 AND 1999;
```

PREDICADO IS NULL

El operador de comprobación de valor nulo, IS NULL, me va a permitir comprobar un campo es o no nulo. Su sintaxis es:

nombre_columna IS [NOT] NULL

Si se especifica NOT, el predicado es verdadero si nombre_columna no es nulo.

Ejemplo: Muesta todos los datos de las asignaturas que todavía no tienen asignado un profesor.

```
SELECT *
FROM asignatura.
WHERE profesor IS NULL;
```

Ejemplo: Muesta el DNI de los alumnos que ya tienen la nota final asingada en alguna de las asiganturas.

```
SELECT DISTINCT dni
FROM alumno_asignatura
WHERE nota_Final IS NOT NULL;
```

Ojo, hay que tener en cuenta que para manejar el valor NULL, se debe utilizar este predicado IS NULL, jamás se podrá usar valor=NULL o valor <>NULL. Esto último no dará el resultado esperado.

PREDICADO LIKE

El operador de comprobación de patrón, LIKE, me va a permitir seleccionar los registros que cumplan una serie de características especificadas en un patrón. Su sintaxis es:

nombre_columna [NOT] LIKE patrón

Donde patrón puede contener cualquier carácter alfanumérico teniendo en cuenta que los caracteres '%' y '_' son caracteres comodines.

- El carácter comodín '_' equivale a cualquier otro carácter, pero siempre uno, y sólo un carácter.
- El carácter comodín '%' equivale a cualquier conjunto de caracteres, bien sean 0, 1, 2,… caracteres.

Ejemplo: Muesta todos los datos de los profesores que tengan correo ufv.

```
SELECT *
FROM profesor
WHERE email LIKE '%@ufv.es';
```

Ejemplo: Muesta todos los datos de los alumnos cuyo nombre tenga exactamente 5 letras.

```
SELECT *
FROM alumno
WHERE nombre LIKE '_____';
```

2.6. Agrupación de resultados: funciones de agregado y cláusula GROUP BY

Las funciones de agregado permiten realizar operaciones sobre un conjunto de resultados (por ejemplo todos los datos de una columna) y devovler un único valor agregado de todos ellos. Es decir, me van a permitir calcular medias, valores máximos, mínimos, etc.

Las funciones de agregación disponibles en MySQL son:

- AVG: calcula la media aritmética de los valores agrupados. Solamente puede utilizarse en columnas numéricas.
- SUM: calcula la suma de los valores agrupados. Al igual que la media sólo tiene sentido en columnas numéricas.
- MIN: devuelve el valor mínimo de los valores agrupados.
- MAX: devuelve el valor máximo de los valores agrupados.
- COUNT: devuelve el número filas agrupadas.

El proceso que internamente y de manera trasnsparente realiza MySQL cuando se utilizan funciones de agregado es el siguiente:

- Antes de aplicar las funciones de agregado se construyen uno o más grupos de filas.
- La creación de estos grupos se especifica con la cláusula GROUP BY.
- En el caso de que no se utilice esta cláusula GROUP BY, se forma un solo grupo con todas las filas de la tabla que cumplen las condiciones especificadas en la cláusula WHERE.
- A cada uno de estos grupos se les aplica las funciones de agregado, devolviendo por lo tanto una tabla con una sola fila (en el caso de que no se haya especificado cláusula GROUP BY) y tantas columnas como expresiones haya en la cláusula SELECT.

La sintaxis de estas funciones de agregado es:

función_agregado ([DISTINCT] nombre_columna)

Hay que tener en cuenta que si se emplea la opción DISTINCT, en primer lugar se eliminan los valores repetidos de las columnas, y sobre estos valores sin repeticiones se aplica la función de agregado.

También hay que tener en cuenta que en la cláusula SELECT no se puede especificar la opción DISTINCT más de una vez, bien sea dentro de una función de agreado o justo detrás de la palabra reservada SELECT.

Ejemplo: Calcula la media de la nota de la primera evaluación del alumno con DNI 25469874X.

```
SELECT AVG(nota_1Ev) FROM alumno_asignatura
WHERE alumno='25469874X';
```

Ejemplo: Calcula la nota máxima de la primera evaluación en la asignatura DAM_BBDD.

```
SELECT MAX(nota_1Ev) FROM alumno_asignatura
WHERE asignatura = 'DAM_BBDD';
```

En el caso de la función COUNT, se puede utilizar una de estas dos sintaxis:
COUNT(nombre_columna)
COUNT(*)

La diferencia entre poner el nombre de una columna o un *, es que en el primer caso no cuenta los valores nulos para dicha columna, y en el segundo sí (devolverá el número filas que hay en el grupo).

Ejemplo: Calcula el número de alumnos matriculados en la asignatura DAM_BBDD.

```
SELECT COUNT(*) FROM alumno_asignatura
WHERE asignatura = 'DAM_BBDD';
```

Ejemplo: Calcula el número de alumnos matriculados en la asignatura DAM_BBDD, junto con el número ellos que tienen nota asignada en la primera evaluación y cuantos en la segunda.

```
SELECT COUNT(*) AS 'Nº de alumnos matriculados',
        COUNT(nota_1Ev) AS 'Nº de alumnos con nota en la 1ª EV',
        COUNT(nota_2Ev) AS 'Nº de alumnos con nota en la 2ª EV'
FROM alumno_asignatura
WHERE asignatura = 'DAM_BBDD';
```

Como se mencionó anteriormente, antes de aplicar las funciones de agregado se construyen uno o más grupos de filas de acuerdo a lo especificado en la cláusula GROUP BY. Por lo tanto, esta cláusuala opcional me permite agrupar las disntintas filas que cumplen las condiciones especificadas en la cláusula WHERE.

A continuación se muestra la sintaxis de la sentencia SELECT con una cláusula GROUP BY. Ojo, es importante destacar que el orden de las diferentes cláusulas dentre del SELECT es el marcado en la sintaxis, por lo tanto la cláusula GROUP BY, irá justo después de la cláusula WHERE.

```
SELECT [ALL | DISTINCT ] expr [, expr...]
[FROM tabla
[WHERE condiciones]
[GROUP BY { nombre_columna | expresión | posición } [ASC | DESC ] ,... [WITH
ROLLUP]]
[ORDER BY { nombre_columna | posición_columna} [ASC | DESC ] ,...]
[LIMIT [desplazamiento, ] num_filas ]
];
```

La cláusula GROUP BY determina que las filas de la tabla que tengan igual valor en las columnas de agrupamiento, indicadas en esta cláusula, pasarán a formar un mismo grupo. Nada impide por lo tanto que existan grupos con una sola fila.

Una vez formados los grupos, se calculan las expresiones de la cláusula SELECT, por lo que la tabla resultante de la consulta tendrá tantas filas como grupos se hayan formado, y tantas columnas como expresiones se indiquen en la cláusula SELECT.

Importantísimo destacar que las columnas que participen en las expresiones de la cláusula SELECT y no sean de agrupamiento (es decir, no sean las incluidas en la cláusula GROUP BY), deben estar incluidas siempre en una función de agregado, sino la consulta será errónea

Ejemplo: Muestra el número de asignaturas impartidas por cada profesor:

```
SELECT profesor, COUNT(*) FROM asignatura
GROUP BY profesor;
```

En este ejemplo, MySQL formará tantos grupos como profesores distintos haya. Es decir la tabla asignatura la voy a dividir en diferentes grupos, de tal manera que cada grupo tendrá siempre el mismo valor en la columna profesor. En nuestro caso hay 3 profesores en la tabla asignatura ('45124785A', '63254157C' y '74512365X'), por lo tanto tendré 3 grupos, el primero formado por las 3 filas con los datos de las asignaturas que imparte el profesor '45124785A', el segundo formado por las 3 filas con los datos de las asignaturas que imparte '63254157C', y el tercero formado por las 2 filas que imparte el profesor '74512365X'. A estos grupos es a los que aplicaré las expresiones indicadas en la cláusula SELECT.

Código	Nombre	Descripción	Horas	Ciclo	Curso	Profesor
DAM_DI	Desarrollo de interfaces	Interfaces	250	DAM	2	45124785A
DAM_PR	Programación	Introducción a la Programación	300	DAM	1	45124785A
DAM_SI	Sistemas informáticos	Sistemas...	210	DAM	1	45124785A
AYF_OYPI	Ofimática y proceso de la información	Ofimática...	250	AYF	1	63254157C
DAM_AC	Acceso de datos	Continuación de bases de datos	250	DAM	2	63254157C
DAM_BBDD	Bases de datos	Introducción a las bases de datos	250	DAM	1	63254157C
AYF_FOL	Formación y orientación laboral	Formación...	150	AYF	1	74512365X
DAM_FOL	Formación y orientación laboral	Formación...	150	DAM	1	74512365X
AYF_CAC	Comunicación y atención al cliente	Se estudiará...	200	AYF	1	NULL
AYF_CF	Contabilidad y Fiscalidad	Se estudiará...	120	AYF	2	NULL
AYF_ING	Inglés	Se estudiará...	160	AYF	1	NULL

Ejemplo: Muestra el número de asignaturas impartidas por profesor en el ciclo formativo DAM.

```
SELECT profesor, COUNT(*) FROM asignatura
WHERE ciclo='DAM'
GROUP BY profesor;
```

Ejemplo: Muestra la nota media en la primera evaluación de cada asignatura, así como la nota máxima y la nota mínima de la clase. Muestra también el número de alumnos matriculados en cada una de ellas.

```
SELECT asignatura, AVG (nota_1Ev) AS 'Nota media en 1Ev',
        MAX(nota_1Ev) AS 'Nota máxima en 1Ev',
        MIN(nota_1Ev) AS 'Nota mínima en 1Ev',
        COUNT(*) AS 'Nº alumnos matriculados'
FROM alumno_asignatura
GROUP BY asignatura;
```

La siguiente consulta sería errónea ya que alumno no es una columna de agrupamiento (no está incluida en la cláusula GROUP BY), ni está incluida en una función de agregado. Aunque aparentemente puede parecer que está bien ya que al ejecutar la consulta no me da ningún error, el valor que da en la columna alumno no tienen ningún sentido, simplemente es uno de los alumnos matriculados en la asignatura que forma ese grupo.

```
SELECT asignatura, alumno, COUNT(*) AS 'Nº alumnos matriculados',
        AVG (nota_1Ev) AS 'Nota media en 1Ev'
FROM alumno_asignatura
GROUP BY asignatura;
```

2.7. Filtro de grupos: cláusula HAVING

La cláusula HAVING filtra los resultados obtenidos por la cláusula GROUP BY, de modo similar a como la cláusula WHERE filtra las filas obtenidas por la cláusula FROM. Si incluimos la cláusula HAVING en la sentencia SELECT, la sintaxis quedaría:

SELECT [ALL | DISTINCT] expresión [, expresión...]
[
FROM tablas
[WHERE condiciones]
[GROUP BY { nombre_columna | expresión | posición } [ASC | DESC] ,... [WITH ROLLUP]]
[HAVING condiciones]
[ORDER BY { nombre_columna | posición_columna} [ASC | DESC] ,...]
[LIMIT [desplazamiento,] num_filas]
];

Por lo tanto, los pasos que va a realizar internamente MySQL en la ejecución de una consulta, en la que partimos de la tabla especificada en la cláusula FROM son:

- En primer lugar se descartan las filas que no cumplan las condiciones establecidas en la cláusula WHERE.
- En segundo lugar se forman grupos tal y como se indique en la cláusula GROUP BY.
- Después de haber formado los grupos, se descartarán aquellos grupos que no cumplan las condiciones establecidas en la cláusula HAVING.
- Y finalmente, sobre los registros que hayan pasado todos estos filtros, se mostrarán como resultado de la consulta las expresiones indicadas en la cláusula SELECT, ordenadas y limitando el número de registros tal y como se indica en las cláusulas ORDER BY y LIMIT.

Estas condiciones que aparecen en la cláusula HAVING, serán condiciones como las que vimos en la cláusula WHERE (formada por operandos, operadores y paréntesis), pero teniendo en cuenta que las columnas que participen en esta condición y que no sean de columnas de agrupamiento, deben de estar siempre incluidas en una función de agregado.

La forma de agrupar filas en grupos se especifica en la cláusula GROUP BY, pero si esta cláusula no existe, se formará un solo grupo con todas las filas de la tabla.

Ejemplo: Muestra el código de las asignaturas cuya nota media en la primera evaluación sea mayor de 7.

```
SELECT asignatura, AVG (nota_1Ev) AS 'Nota media en 1Ev'
FROM alumno_asignatura
GROUP BY asignatura
HAVING AVG(nota_1Ev)>7;
```

Ejemplo: Muestra el código de las asignaturas con mas de 3 alumnos matriculados.

```
SELECT asignatura, COUNT(*) AS 'Nº alumnos matriculados'
FROM alumno_asignatura
GROUP BY asignatura
HAVING COUNT(*)>3;
```

03. Subconsultas

Muchas veces el valor de la condición de un predicado de la cláusula WHERE O HAVING no se conoce de antemano y es necesario realizar otra consulta para comprobar su valor. Las subconsultas evitan tener que realizar dos consultas separadas.

Ejemplo: Muestra los nombres, curso y ciclo formativo al que pertenecen las asignaturas impartidas por el profesor cuyo email es “esmeralda.sanz@ufv.es”.

La tabla asignaturas sólo tiene el DNI del profesor que imparte cada asignatura, por lo que necesitaré obtener el DNI del profesor a partir de su email en la tabla profesor.

```
SELECT dni FROM profesor
WHERE email = "esmeralda.sanz@ufv.es";
```

El resultado de esta consulta es '63254157C'. Con este dato ya puedo seleccionar los datos pedidos de las asignaturas impartidas por dicho profesor.

```
SELECT nombre, curso, ciclo FROM asignatura
WHERE profesor= '63254157C';
```

Esto mismo lo puedo realizar en una sola sentencia SELECT usando subconsultas, para ello, solamente tengo que sustituir el DNI del profesor en la segunda consulta ('63254157C') por la SELECT que he tenido que ejecutar para obtener ese resultado.

```
SELECT nombre, curso, ciclo FROM asignatura
WHERE profesor= (SELECT dni FROM profesor
                 WHERE email = "esmeralda.sanz@ufv.es";);
```

Por lo tanto, una subconsulta es una sentencia SELECT que se sitúa como segundo operando en una condición de una cláusula WHERE o HAVING, en la que se deben de tener en cuenta ciertos aspectos:

- La subconsulta debe ir entre paréntesis.
- El resultado de la subconsulta debe estar compuesto por una única columna.
- Una subconsulta puede contener a su vez otras subconsultas.

A continuación, se detallan los posibles operadores que pueden ser usados con las subconsultas:

- Operadores de comparación.
- Operador de pertenencia a conjunto: IN.
- Operadores cuantificados: ALL y ANY.
- Operador de existencia: EXISTS.

3.1. Predicados de comparación

En estos predicados se utilizan los operadores de comparación (>, <, >=, <=, <> o !=, !>, !<) para comparar un operando con el resultado generado por una subconsulta. Si sintaxis es:

expresión operador_comparación (subconsulta)

Las subconsultas deben devolver siempre una única columna, pero si estamos utilzando un operador de comparación deberá además devolver una única fila. Es decir, un solo registro (una única fila y una única columna).

Ejemplo: Muestra los datos de los alumnos que hayan nacido después del alumno con email 'david.moreno@yahoo.es'.

```
SELECT *
FROM alumno
WHERE fecha_nacimiento > (SELECT fecha_nacimiento FROM alumno
                          WHERE email= 'david.moreno@yahoo.es');
```

3.2. Predicado de pertenencia a un conjunto: IN

El operador IN, al igual que cuando lo utilizábamos en consultas simples, me va a permitir comprobar si el resultado de una expresión está incluido en el conjunto de valores especificados entre paréntesis tras la palabra reservada IN. Este conjunto de valores son los que se obtienen con una subconsulta. Su sintaxis es:

expresión [NOT] IN (subconsulta)

En este caso se permite que la subconsulta devuelva varias filas, pero en una única columna.

Ejemplo: Muestra el nombre y apellido de los alumnos que hayan aprobado la asignatura cuyo cóidgo es DAM_BBDD. El resultado se deberá mostrar ordenado alfabéticamente por apellido y para alumnos con el mismo apellido ordenados por nombre.

```
SELECT nombre, apellido
FROM alumno
WHERE dni IN (SELECT alumno FROM alumno_asignatura
        WHERE nota_1Ev >= 5 AND asignatura ='DAM_BBDD')
ORDER BY apellido, nombre;
```

Ejemplo: Normalmente, el dato proporcionado en el ejemplo anterior no sería el código de la asignatura, sino el nombre y el ciclo de la misma, es decir, el enunciado sería: "muestra el nombre y apellido de los alumnos que hayan aprobado la asignatura de Bases de datos de DAM. El resultado se deberá mostrar ordenado …". En este caso, habría que añadir otra subconsulta a la subconsulta.

```
SELECT nombre, apellido FROM alumno
WHERE dni IN (SELECT alumno FROM alumno_asignatura
        WHERE nota_1Ev >= 5 AND
        asignatura =(SELECT codigo FROM asignatura
                WHERE nombre='Bases de datos' AND ciclo='DAM' ))
ORDER BY apellido, nombre;
```

3.3. Predicados cuantificados: ALL y ANY

Los operandos cuantificados ALL y ANY me van a permitir calcular la relación entre una expresión y todos los registros de la subconsulta (ALL) o algunos de los registros de la subconsulta (ANY). Su sintaxis es:

expresión operador_comparación { ALL | ANY} (subconsulta)

De esta manera:

- El predicado ALL es verdadero si la comparación es cierta para todos los valores devueltos por la subconsulta.
- El predicado ANY es verdadero si la comparación es cierta para al menos uno de los valores devueltos por la subconsulta.

Al igual que ocurría en el predicado IN, en este caso se permite que la subconsulta devuelva varias filas, pero en una única columna.

Ejemplo: Muestra los datos de las asignaturas de segundo curso que tienen más carga lectiva que todas las asignaturas de primer curso en el ciclo formativo DAM.

```
SELECT * FROM asignatura
WHERE curso=2 AND ciclo ='DAM' AND
horas > ALL(SELECT horas FROM asignatura
            WHERE curso=1 AND ciclo ='DAM');
```

Ejemplo: Muestra los datos de las asignaturas de segundo curso que tienen más carga lectiva que alguna de las asignaturas de primer curso en el ciclo formativo DAM.

```
SELECT * from asignatura
WHERE curso=2 AND ciclo ='DAM' AND
horas > ANY(SELECT horas FROM asignatura
            WHERE curso=1 AND ciclo ='DAM');
```

3.4. Predicados de existencia: EXISTS

El predicado EXISTS, con la palabra reservada opcional NOT, se utiliza en comparaciones de verdadero o falso para determinar si la subconsulta devuelve algún registro. De esta manera, si una subconsulta devuelve cualquier fila (al menos una), el predicado:

- EXISTS (subconsulta) será verdadero.
- NOT EXISTS (subconsulta) será falso.

Por ejemplo:

```
SELECT columna1 FROM t1 WHERE EXISTS (SELECT * FROM t2);
```

Normalmente, la subconsulta que sigue a un operador EXISTS empieza por "SELECT *", pero podría empezar por "SELECT 5" o "SELECT columna1". MySQL ignora la lista de expresiones incluidas en la cláusula SELECT de la consulta, por lo que no hay diferencia en poner una cosa u otra.

En el ejemplo anterior, si la tabla t2 contiene cualquier fila, el predicado EXISTS es verdadero por lo que se mostrarán las filas de la tabla t1.

Se ha puesto este ejemplo genérico para que se entienda a modo global cómo funciona el predicado EXISTS ya que, en la realidad, las subconsultas con cláusulas EXISTS, contienen casi siempre subconsultas correlacionadas que se estudiarán en el siguiente apartado. En ese apartado pondremos ejemplos más realistas con la cláusula EXISTS.

04. Consultas correlacionadas

Hasta el momento, todas las subconsultas realizadas se pueden ejecutar independientemente de las consultas antecedentes. El SGBD ejecutaba cada subconsulta una sola vez, y sustituía su valor en el predicador donde se encontraba. Esto es debido a que las subconsultas anidadas no hacen referencia a columnas de tablas que no estén en su propia cláusula FROM.

En las consultas correlacionadas no se va a poder ejecutar la subconsulta independientemente, ya que va a hacer referencia a alguna columna de alguna tabla especificada en la cláusula FROM de la consulta antecedente.

En este caso, el proceso que sigue el SGBD para la ejecución de la consulta es:

- La consulta externa pasa cada una de sus filas a la subconsulta.
- La subconsulta se evalúa gracias a los valores proporcionados por la consulta externa y le devolverá las filas que cumplan las condiciones especificadas en sus cláusulas WHERE y HAVING
- Este proceso
- se repetirá para cada una de las filas de la consulta externa.

En este tipo de consultas se hace referencia a columnas de más de una tabla. En dos tablas distintas las columnas pueden tener el mismo nombre, por lo que es imprescindible indicar a que atributo de qué tabla me estoy refiriendo. Por ejemplo para referirnos al nombre de un profesor escribiré profesor.nombre, mientras que para referirnos al nombre de una asignatura tendré que indicar asignatura.nombre. Muchas veces para simplificar su escritura, se utilizarán alias para las tablas, como se explicó al principio del tema.

Ejemplo: Muestra los datos de las asignaturas con más de 3 alumnos matriculados.

```
SELECT * FROM asignatura a
WHERE 3 < (SELECT COUNT(*) FROM alumno_asignatura aa
           WHERE aa.asignatura=a.codigo);
```

El proceso que sigue el SGBD en este ejemplo es:

- La consulta externa va a pasar la primera fila ('AYF_FOL', 'Formación y orientación laboral', 'Formación...', 150, 'AYF', 1, '74512365X') a la subconsulta.

- Con estos datos la subconsulta se ejecutará, en este caso:

```
SELECT COUNT(*) FROM alumno_asignatura aa
WHERE aa.asignatura='AYF_FOL';
```

- El resultado de la ejecución de esta consulta es 2, por lo que 2 no es mayor que 3, y por lo tanto esta asignatura no será mostrada como resultado final de la consulta correlacionada.

- A continuación la consulta externa pasará a la subconsulta la segunda fila ('AYF_OYPI', 'Ofimática y proceso de la información', 'Ofimática...', '250', 'AYF', '1', '63254157C').

- La subconsulta que se ejecutará con estos datos es:

```
SELECT COUNT(*) FROM alumno_asignatura aa
WHERE aa.asignatura='AYF_OYPI';
```

- El resultado de la ejecución de esta consulta también es 2, por lo que esta asignatura tampoco será mostrada como resultado final de la consulta correlacionada.

- A continuación la consulta externa pasará a la subconsulta la segunda fila ('DAM_AC', 'Acceso de datos', 'Continuación de bases de datos', '250', 'DAM', '2', '63254157C').

- La subconsulta que se ejecutará con estos datos es:

```
SELECT COUNT(*) FROM alumno_asignatura aa
WHERE aa.asignatura='DAM_AC';
```

- El resultado de la ejecución de esta consulta también es 3, por lo que…

Este proceso se repetirá hasta que la consulta externa haya pasado todas las filas de su tabla a la subconsulta.

Ejemplo: Muestra los datos de las asignaturas donde no haya ningún alumno matriculado.

```
SELECT * FROM asignatura a
WHERE NOT EXISTS (SELECT * FROM alumno_asignatura aa
                  WHERE aa.asignatura=a.codigo);
```

05. Consultas con tablas derivadas

Las consultas con tablas derivadas son aquellas que en su cláusula FROM no van a tener un nombre de una tabla, sino una sentencia SELECT. Por tanto, esta consulta SELECT pasará a ser una tabla derivada.

Ejemplo: muestra el ciclo formativo con mayor número de horas lectivas.

```
SELECT MAX(tderivada.horas_ciclo)
FROM (SELECT SUM(horas) AS 'horas_ciclo', ciclo
        FROM asignatura GROUP BY (ciclo)) AS tderivada;
```

En este ejemplo habrá que calcular en primer lugar el número de horas totales por ciclo. Eso se hace de la siguiente manera:

```
SELECT SUM(horas) AS 'Horas_ciclo', ciclo
FROM asignatura
GROUP BY (ciclo)
```

La ejecución de esta consulta devuelve una tabla formada por dos columnas, horas_ciclo y ciclo. Ésta será la tabla por tanto que voy a incluir en la cláusula FROM asignándole un alias (tderivada).

06. Consultas sobre varias tablas: JOIN

En muchas ocasiones, las consultas necesitan obtener información de más de una tabla, bien sea porque los campos a mostrar están en una tabla diferente (por ejemplo, muestra el nombre del profesor que imparte cada asignatura) o bien porque las condiciones impuestas afectan a campos de otra tabla (Por ejemplo, muestra todos los datos de los profesores que imparten alguna asignatura en el ciclo formativo DAM).

En estos casos, podemos incluir todas las tablas necesarias en la consulta en la cláusula FROM. Esto va a provocar el producto cartesiano de las tablas, lo que es

lo mismo, generará todas las combinaciones posibles de las filas de una tabla con las filas de la otra tabla, como muestra la imagen siguiente.

```
1 •  SELECT * FROM asignatura, profesor;
```

esult Grid | Filter Rows: | Export: | Wrap Cell Content:

codigo	nombre	descripcion	horas	ciclo	curso	profesor	dni	nombre	apellido	email	telefono	direccion
AYF_CAC	Comunicación y atención ...	Se estudiará...	200	AYF	1	NULL	45124785A	Jorge	Alonso	jorge.alonso@ufv...	965875487	Calle Altamira, 25
AYF_CAC	Comunicación y atención ...	Se estudiará...	200	AYF	1	NULL	63254157C	Esmeralda	Sanz	esmeralda.sanz@...	658784578	Calle Protugal, 3,
AYF_CAC	Comunicación y atención ...	Se estudiará...	200	AYF	1	NULL	65847512N	Marta	Potro	marta.potro@ufv.es	652148754	Calle Gran via, 17
AYF_CAC	Comunicación y atención ...	Se estudiará...	200	AYF	1	NULL	74512365X	Sergio	Prieto	mario.blanco@ya...	965874521	Calle Fuencarral,
AYF_CF	Contabilidad y Fiscalidad	Se estudiará...	120	AYF	2	NULL	45124785A	Jorge	Alonso	jorge.alonso@ufv...	965875487	Calle Altamira, 25
AYF_CF	Contabilidad y Fiscalidad	Se estudiará...	120	AYF	2	NULL	63254157C	Esmeralda	Sanz	esmeralda.sanz@...	658784578	Calle Protugal, 3,
AYF_CF	Contabilidad y Fiscalidad	Se estudiará...	120	AYF	2	NULL	65847512N	Marta	Potro	marta.potro@ufv.es	652148754	Calle Gran via, 17
AYF_CF	Contabilidad y Fiscalidad	Se estudiará...	120	AYF	2	NULL	74512365X	Sergio	Prieto	mario.blanco@ya...	965874521	Calle Fuencarral,
AYF_FOL	Formación y orientación la...	Formación...	150	AYF	1	74512365X	45124785A	Jorge	Alonso	jorge.alonso@ufv...	965875487	Calle Altamira, 25
AYF_FOL	Formación y orientación la...	Formación...	150	AYF	1	74512365X	63254157C	Esmeralda	Sanz	esmeralda.sanz@...	658784578	Calle Protugal, 3,
AYF_FOL	Formación y orientación la...	Formación...	150	AYF	1	74512365X	65847512N	Marta	Potro	marta.potro@ufv.es	652148754	Calle Gran via, 17
AYF_FOL	Formación y orientación la...	Formación...	150	AYF	1	74512365X	74512365X	Sergio	Prieto	mario.blanco@ya...	965874521	Calle Fuencarral,
AYF_ING	Inglés	Se estudiará...	160	AYF	1	NULL	45124785A	Jorge	Alonso	jorge.alonso@ufv...	965875487	Calle Altamira, 25
AYF_ING	Inglés	Se estudiará...	160	AYF	1	NULL	63254157C	Esmeralda	Sanz	esmeralda.sanz@...	658784578	Calle Protugal, 3,
AYF_ING	Inglés	Se estudiará...	160	AYF	1	NULL	65847512N	Marta	Potro	marta.potro@ufv.es	652148754	Calle Gran via, 17
AYF_ING	Inglés	Se estudiará...	160	AYF	1	NULL	74512365X	Sergio	Prieto	mario.blanco@ya...	965874521	Calle Fuencarral,
AYF_OYPI	Ofimática y proceso de la ...	Ofimática...	250	AYF	1	63254157C	45124785A	Jorge	Alonso	jorge.alonso@ufv...	965875487	Calle Altamira, 25
AYF_OYPI	Ofimática y proceso de la ...	Ofimática...	250	AYF	1	63254157C	63254157C	Esmeralda	Sanz	esmeralda.sanz@...	658784578	Calle Protugal, 3,
AYF_OYPI	Ofimática y proceso de la ...	Ofimática...	250	AYF	1	63254157C	65847512N	Marta	Potro	marta.potro@ufv.es	652148754	Calle Gran via, 17
AYF_OYPI	Ofimática y proceso de la ...	Ofimática...	250	AYF	1	63254157C	74512365X	Sergio	Prieto	mario.blanco@ya...	965874521	Calle Fuencarral,
DAM_AC	Acceso de datos	Continuación de ...	250	DAM	2	63254157C	45124785A	Jorge	Alonso	jorge.alonso@ufv...	965875487	Calle Altamira, 25

Como se puede observar, cada una de las asignaturas es combinada con todos los profesores, pero muchos de estos datos no tienen sentido, por ejemplo, sí tiene sentido que combine la asignatura AYF_FOL con el profesor Sergio Prieto, que es el que la imparte, pero no tiene ningún sentido el resto de las combinaciones. Estas filas sin sentido se denominan filas espurias y deben de ser eliminadas.

Para eliminar filas espurias basta con igualar los atributos que relacionan ambas tablas (es decir la clave ajena). De esta manera, en nuestro ejemplo debo igualar el campo profesor de la tabla asignatura con el campo DNI de la tabla profesor.

Puede darse el caso de que dos atributos de distintas tablas tengan el mismo nombre, esta posible ambigüedad la puedo eliminar de dos maneras:

- Cualificando el atributo con el nombre de la tabla, es decir, profesor.nombre.
- cualificando el atributo con un alias creado sobre la tabla, de tal manera que si el alias de la tabla profesor es 'p', pondré, p.nombre.

De esta manera, nuestro ejemplo quedaría:

```
1 •  SELECT * FROM asignatura a, profesor p
2    WHERE a.profesor=p.DNI;
```

sult Grid | Filter Rows: | Export: | Wrap Cell Content:

codigo	nombre	descripcion	horas	ciclo	curso	profesor	dni	nombre	apellido	email	telefono	direccion
AYF_FOL	Formación y orientación la...	Formación...	150	AYF	1	74512365X	74512365X	Sergio	Prieto	mario.blanco@ya...	965874521	Calle Fuencarral,
AYF_OYPI	Ofimática y proceso de la ...	Ofimática...	250	AYF	1	63254157C	63254157C	Esmeralda	Sanz	esmeralda.sanz@...	658784578	Calle Protugal, 3,
DAM_AC	Acceso de datos	Continuación de ...	250	DAM	2	63254157C	63254157C	Esmeralda	Sanz	esmeralda.sanz@...	658784578	Calle Protugal, 3,
DAM_BBDD	Bases de datos	Introducción a la...	250	DAM	1	63254157C	63254157C	Esmeralda	Sanz	esmeralda.sanz@...	658784578	Calle Protugal, 3,
DAM_DI	Desarrollo de interfaces	Interfaces	250	DAM	2	45124785A	45124785A	Jorge	Alonso	jorge.alonso@ufv...	965875487	Calle Altamira, 25
DAM_FOL	Formación y orientación la...	Formación...	150	DAM	1	74512365X	74512365X	Sergio	Prieto	mario.blanco@ya...	965874521	Calle Fuencarral,
DAM_LM	Lenguaje de Marcas	XML, HTML, ...	140	DAM	1	63254157C	63254157C	Esmeralda	Sanz	esmeralda.sanz@...	658784578	Calle Protugal, 3,
DAM_PR	Programación	Introducción a la...	300	DAM	1	45124785A	45124785A	Jorge	Alonso	jorge.alonso@ufv...	965875487	Calle Altamira, 25
DAM_SI	Sistemas informáticos	Sistemas...	210	DAM	1	45124785A	45124785A	Jorge	Alonso	jorge.alonso@ufv...	965875487	Calle Altamira, 25

En este ejemplo también llama la atención que, si en la cláusula SELECT especifico un *, se mostrará todos los atributos de todas las tablas incluidas en la cláusula FROM, y en el mismo orden. Del mismo modo que ocurría en consultas de una sola tabla en la cláusula SELECT puedo especificar atributos sueltos de cualquiera de las tablas.

```
1 •  SELECT a.codigo, a.nombre, a.ciclo, a.curso, p.nombre, p.apellido
2    FROM asignatura a, profesor p
3    WHERE a.profesor=p.DNI;
```

sult Grid | Filter Rows: | Export: | Wrap Cell Content:

codigo	nombre	ciclo	curso	nombre	apellido
AYF_FOL	Formación y orientación la...	AYF	1	Sergio	Prieto
AYF_OYPI	Ofimática y proceso de la ...	AYF	1	Esmeralda	Sanz
DAM_AC	Acceso de datos	DAM	2	Esmeralda	Sanz
DAM_BBDD	Bases de datos	DAM	1	Esmeralda	Sanz
DAM_DI	Desarrollo de interfaces	DAM	2	Jorge	Alonso
DAM_FOL	Formación y orientación la...	DAM	1	Sergio	Prieto
DAM_LM	Lenguaje de Marcas	DAM	1	Esmeralda	Sanz
DAM_PR	Programación	DAM	1	Jorge	Alonso
DAM_SI	Sistemas informáticos	DAM	1	Jorge	Alonso

Si quiero mostrar todos los datos de una de las tablas y solamente algún que otro atributo de la otra tabla, puedo "cualificar el *", es decir, indicar delante del asterisco, el nombre de la tabla o su alias de la que deseo mostrar todos sus datos. Por ejemplo:

```
1 • SELECT a.*, p.nombre, p.apellido
2   FROM asignatura a, profesor p
3   WHERE a.profesor=p.DNI;
```

esult Grid | Filter Rows: | Export: | Wrap Cell Content:

codigo	nombre	descripcion	horas	ciclo	curso	profesor	nombre	apellido
AYF_FOL	Formación y orientación la...	Formación...	150	AYF	1	74512365X	Sergio	Prieto
AYF_OYPI	Ofimática y proceso de la ...	Ofimática...	250	AYF	1	63254157C	Esmeralda	Sanz
DAM_AC	Acceso de datos	Continuación de ...	250	DAM	2	63254157C	Esmeralda	Sanz
DAM_BBDD	Bases de datos	Introducción a la...	250	DAM	1	63254157C	Esmeralda	Sanz
DAM_DI	Desarrollo de interfaces	Interfaces	250	DAM	2	45124785A	Jorge	Alonso
DAM_FOL	Formación y orientación la...	Formación...	150	DAM	1	74512365X	Sergio	Prieto
DAM_LM	Lenguaje de Marcas	XML, HTML, ...	140	DAM	1	63254157C	Esmeralda	Sanz
DAM_PR	Programación	Introducción a la...	300	DAM	1	45124785A	Jorge	Alonso
DAM_SI	Sistemas informáticos	Sistemas...	210	DAM	1	45124785A	Jorge	Alonso

En resumen:

- Un JOIN es una consulta que combina datos de varias tablas.
- La cláusula FROM incluirá todas las tablas implicadas.
- Conceptualmente, la consulta se realiza sobre una tabla nueva que sería el resultado de aplicar el producto cartesiano a todas las tablas incluidas en la cláusula FROM.
- La cláusula WHERE debe incluir las condiciones de igualdad que relacionan la tablas de la cláusula FROM.

Es importantísimo no olvidar eliminar las filas espurias. Siempre que en la cláusula FROM haya más de una tabla, automáticamente en la clásula WHERE eliminaré las filas espurias igualando las condiciones que relacionan esas tablas.

La sintaxis habitual de un JOIN, es la que se ha mostrado en los ejemplos:

- Las tablas incluidas en la cláusula FROM son separadas por comas, y
- Las filas espurias se eliminan como una condición más en la cláusula WHERE.

Sin embargo, es posible utilizar dos sintaxis más. La primera de ellas es:

- Las tablas incluidas en la cláusula FROM son separadas por la palabra reservada JOIN o INNER JOIN, y
- Las filas espurias se eliminan como una condición más en la cláusula WHERE.

La última sintaxis posible es:

- Las tablas incluidas en la cláusula FROM son separadas por la palabra reservada JOIN, y
- Las filas espurias se eliminan igualando las condiciones que relacionan esas tablas en la cláusula ON.

Ejemplo: Muestra las notas en la 1ª evaluación de DAM_BBDD de todos los alumnos. Se deberá mostrar el nombre y apellido de cada uno de los alumnos, junto con su nota. (Estas tres consultas generan el mismo resultado).

```
SELECT a.nombre, a. apellido, aa.nota_1Ev
FROM alumno a, alumno_asignatura aa
WHERE a.dni=aa.alumno AND aa.asignatura='DAM_BBDD';

SELECT a.nombre, a. apellido, aa.nota_1Ev
FROM alumno a JOIN alumno_asignatura aa
WHERE a.dni=aa.alumno AND aa.asignatura='DAM_BBDD';

SELECT a.nombre, a. apellido, aa.nota_1Ev
FROM alumno a JOIN alumno_asignatura aa
ON a.dni=aa.alumno AND aa.asignatura='DAM_BBDD';
```

6.1. Composición interna: INNER JOIN

Es el JOIN tal y como has sido explicado anteriormente. En este caso las filas que tienen valores nulos en los campos comunes, no son combinadas, y por tanto no aparecerán nunca en en restultado final.

Ejemplo: Muestra todos los datos de las asignaturas, junto con el nombre y el apellido del profesor que la imparte.

```
SELECT a.*, p.nombre, p.apellido
FROM asignatura a INNER JOIN profesor p
WHERE a.profesor=p.DNI;
```

Esta consulta genera el resultado que se muestra en la siguiente imagen:

```
1  SELECT a.*, p.nombre, p.apellido
2  FROM asignatura a INNER JOIN profesor p
3  WHERE a.profesor=p.DNI;
4
```

esult Grid | Filter Rows: | Export: | Wrap Cell Content:

codigo	nombre	descripcion	horas	ciclo	curso	profesor	nombre	apellido
AYF_FOL	Formación y orientación la...	Formación...	150	AYF	1	74512365X	Sergio	Prieto
AYF_OYPI	Ofimática y proceso de la ...	Ofimática...	250	AYF	1	63254157C	Esmeralda	Sanz
DAM_AC	Acceso de datos	Continuación de ...	250	DAM	2	63254157C	Esmeralda	Sanz
DAM_BBDD	Bases de datos	Introducción a la...	250	DAM	1	63254157C	Esmeralda	Sanz
DAM_DI	Desarrollo de interfaces	Interfaces	250	DAM	2	45124785A	Jorge	Alonso
DAM_FOL	Formación y orientación la...	Formación...	150	DAM	1	74512365X	Sergio	Prieto
DAM_LM	Lenguaje de Marcas	XML, HTML, ...	140	DAM	1	63254157C	Esmeralda	Sanz
DAM_PR	Programación	Introducción a la...	300	DAM	1	45124785A	Jorge	Alonso
DAM_SI	Sistemas informáticos	Sistemas...	210	DAM	1	45124785A	Jorge	Alonso

Como se observa, las asignaturas que no tienen profesor asignado no se mostrarán en el resultado.

6.2. Composición externa: OUTER JOIN

En el OUTER JOIN se combinan absolutamente todas las finas, aunque tengan valores nulos en los campos comunes. Cabe destacar que en este tipo de consultas la eliminación de filas espurias se debe de hacer siempre en la cláusula ON, nunca se podrá hacer en el WHERE.

Existen dos tipos de OUTER JOIN:

- LEFT JOIN: en este caso se combinan todas las filas de la primera tabla incluída en la cláusula FROM con las filas de la segunda tabla que cumplan la condición especificada en la cláusula ON, es decir, la combinación se hace de izquierda a derecha.
- RIGHT JOIN: en este caso se combinan todas las filas de la segunda tabla incluída en la cláusula FROM con las filas de la primera tabla que cumplan la condición especificada en la cláusula ON, es decir, la combinación se hace de izquierda a derecha.

Ejemplo: Muestra todos los datos de las asignaturas, junto con el nombre y el apellido del profesor que la imparte. Se debe mostrar todos los datos de todas las asignaturas, incluso de aquellas que no tienen profesor asignado.

```
SELECT a.*, p.nombre, p.apellido
FROM asignatura a LEFT JOIN profesor p
ON a.profesor=p.DNI;
```

Esta consulta genera el resultado que se muestra en la siguiente imagen:

```
1 • SELECT a.*, p.nombre, p.apellido
2   FROM asignatura a LEFT JOIN profesor p
3   ON a.profesor=p.DNI;
4
```

esult Grid | Filter Rows: | Export: | Wrap Cell Content:

codigo	nombre	descripcion	horas	ciclo	curso	profesor	nombre	apellido
DAM_DI	Desarrollo de interfaces	Interfaces	250	DAM	2	45124785A	Jorge	Alonso
DAM_PR	Programación	Introducción a la...	300	DAM	1	45124785A	Jorge	Alonso
DAM_SI	Sistemas informáticos	Sistemas...	210	DAM	1	45124785A	Jorge	Alonso
AYF_OYPI	Ofimática y proceso de la ...	Ofimática...	250	AYF	1	63254157C	Esmeralda	Sanz
DAM_AC	Acceso de datos	Continuación de ...	250	DAM	2	63254157C	Esmeralda	Sanz
DAM_BBDD	Bases de datos	Introducción a la...	250	DAM	1	63254157C	Esmeralda	Sanz
DAM_LM	Lenguaje de Marcas	XML, HTML, ...	140	DAM	1	63254157C	Esmeralda	Sanz
AYF_FOL	Formación y orientación la...	Formación...	150	AYF	1	74512365X	Sergio	Prieto
DAM_FOL	Formación y orientación la...	Formación...	150	DAM	1	74512365X	Sergio	Prieto
AYF_CAC	Comunicación y atención ...	Se estudiará...	200	AYF	1	NULL	NULL	NULL
AYF_CF	Contabilidad y Fiscalidad	Se estudiará...	120	AYF	2	NULL	NULL	NULL
AYF_ING	Inglés	Se estudiará...	160	AYF	1	NULL	NULL	NULL

Ejemplo: Muestra todos los datos de las asignaturas, junto con el nombre y el apellido del profesor que la imparte. Se debe mostrar todos los datos de todas los profesores, incluso de aquellos que no imparten ninguna asignatura todavía.

```
SELECT a.*, p.nombre, p.apellido
FROM asignatura a RIGHT JOIN profesor p
ON a.profesor=p.DNI;
```

```
1 •  SELECT a.*, p.nombre, p.apellido
2    FROM asignatura a RIGHT JOIN profesor p
3    ON a.profesor=p.DNI;
4
```

sult Grid | Filter Rows: | Export: | Wrap Cell Content:

codigo	nombre	descripcion	horas	ciclo	curso	profesor	nombre	apellido
DAM_DI	Desarrollo de interfaces	Interfaces	250	DAM	2	45124785A	Jorge	Alonso
DAM_PR	Programación	Introducción a la...	300	DAM	1	45124785A	Jorge	Alonso
DAM_SI	Sistemas informáticos	Sistemas...	210	DAM	1	45124785A	Jorge	Alonso
AYF_OYPI	Ofimática y proceso de la ...	Ofimática...	250	AYF	1	63254157C	Esmeralda	Sanz
DAM_AC	Acceso de datos	Continuación de ...	250	DAM	2	63254157C	Esmeralda	Sanz
DAM_BBDD	Bases de datos	Introducción a la...	250	DAM	1	63254157C	Esmeralda	Sanz
DAM_LM	Lenguaje de Marcas	XML, HTML, ...	140	DAM	1	63254157C	Esmeralda	Sanz
NULL	NULL	NULL	NULL	NULL	NULL	NULL	Marta	Potro
AYF_FOL	Formación y orientación la...	Formación...	150	AYF	1	74512365X	Sergio	Prieto
DAM_FOL	Formación y orientación la...	Formación...	150	DAM	1	74512365X	Sergio	Prieto

07. Bibliografía

Hueso Ibañez, L. (2012). Bases de datos. Editorial Ra-Ma.

López Montalbán, I. y de Castro Vázquez, M. (2014). Gestión de bases de datos. Editorial Garceta.

Date, C. J. (2001). Introducción a los sistemas de bases de datos. Editorial Pearson Prentice Hall.

De Miguel, A.; Piattini, M.; Marcos, E. (1999). Diseño de bases de datos relacionales. Editorial Ra-Ma.

(2020). MySQL. SELECT Sintex. Recuperado de: https://dev.mysql.com/doc/refman/5.7/en/select.html.